Lab 12: Lasso, Ridge, Trees, and Neural Networks

SOCIOL 723, Social Statistics II, Thursday, November 12

Wenhao Jiang

Table of contents

1 Goals **2**

2 Penalized regression in three simulated worlds **2**
2.1 Three data-generating processes . . . 2
2.2 Approximate sparsity . . . 3
2.3 Lasso and post-lasso . . . 4
2.4 Ridge, elastic net, and plug-in lasso . . . 5

3 Lasso on the GSS **7**
3.1 Data, design, and the split . . . 7
3.2 The zero-or-not rule, checked against glmnet . . . 7
3.3 Cross-validation by hand . . . 8
3.4 The plug-in penalty by hand . . . 10
3.5 Post-lasso . . . 10
3.6 Lasso selection is unstable . . . 11
3.7 How to do cross-validation wrong . . . 12

4 Trees and ensembles **12**
4.1 A noiseless curve (690S) . . . 12
4.2 One split by hand . . . 14
4.3 Trees on the GSS . . . 15
4.4 Bagging and random forests . . . 16
4.5 Boosting by hand . . . 18

5 Neural networks **19**
5.1 Gradient descent and backpropagation by hand . . . 19
5.2 A network on the noiseless curve . . . 19
5.3 A network on the GSS . . . 21

6 Exercises **21**

7 Session information **21**

1 Goals

1. Fit lasso, post-lasso, ridge, and elastic net in simulated worlds that are sparse, dense, or both, and see which penalty wins where (the 690S Week 2 code).
2. On the GSS: check the lasso's zero-or-not rule by hand, run cross-validation by hand, compute the plug-in penalty by hand, and refit post-lasso.
3. Grow trees, forests, and boosted trees, first on a noiseless curve (the 690S Week 3 code), then on the GSS; do one split and four boosting steps by hand.
4. Fit neural networks; do gradient descent and one backpropagation step by hand.

Every number on the Week 12 slides that is not from `figs/w12_gss.R` comes from this lab or from `figs/w12_690s.R`, which uses the same data, seed, and split.

```
library(glmnet)       ## lasso, ridge, elastic net
library(hdm)          ## lasso with the plug-in penalty (rlasso)
library(ggplot2)
library(ggpubr)       ## ggarrange: panels side by side
library(rpart)        ## regression trees
library(rpart.plot)   ## drawing trees
library(ranger)       ## fast random forests (GSS)
library(randomForest) ## random forests (690S code)
library(gbm)          ## boosted trees (690S code)
library(nnet)         ## one-hidden-layer neural networks
## a plain theme for every figure
theme_lab <- theme_bw(base_size = 10) +
  theme(panel.grid = element_blank(), legend.title = element_blank(),
        legend.position = "bottom")
```

2 Penalized regression in three simulated worlds

2.1 Three data-generating processes

We follow the 690S code. The regressors are uniform on $(-0.5, 0.5)$, the error is standard normal, and the coefficients come in three shapes: **sparse** ($\beta_j = 7/j^2$), **dense** (every β_j small and random), and **sparse and dense** (both at once). With $n = 100$ and $p = 400$, OLS does not exist.

```
## generate one sample, one test sample, and a large "population" from a chosen
DGP
gen_data <- function(n, p, regime = "sparse", npop = 20000) {
  ## constants chosen to give an R^2 of roughly .80
  if (regime == "sparse") {
    beta <- (1 / seq(1:p)^2) * 7                    ## a few big, then tiny
  } else if (regime == "dense") {
```

```
    beta <- rnorm(p) * 0.2                              ## all small, comparable
size
  } else if (regime == "sparsedense") {
    beta <- (1 / seq(1:p)^2) * 6.5 + rnorm(p, 0, 0.5) * 0.7   ## both
  }
  true_fn <- function(x) x %*% beta                     ## the true CEF is linear
  X <- matrix(runif(n * p, -0.5, 0.5), n, p)            ## training regressors
  y <- true_fn(X) + rnorm(n)                            ## training outcome
  Xpop <- matrix(runif(npop * p, -0.5, 0.5), npop, p)  ## fresh cases to test
on
  ypop <- true_fn(Xpop) + rnorm(npop)
  list(X = X, y = as.vector(y), Xpop = Xpop, ypop = as.vector(ypop), beta =
  beta)
}
```

2.2 Approximate sparsity

```
set.seed(1)
n <- 100; p <- 400
res <- gen_data(n, p, regime = "sparse")
X <- res$X; y <- res$y; betas <- res$beta
## plot the 20 largest coefficients
ggplot(data.frame(j = seq_along(betas), magnitude = abs(betas)),
       aes(j, magnitude)) +
  geom_line(colour = "navy", linewidth = 0.4) +
  geom_point(shape = 21, size = 2, colour = "navy", fill = "white") +
  xlim(1, 20) + labs(x = "j", y = "j-th largest coefficient") + theme_lab
```

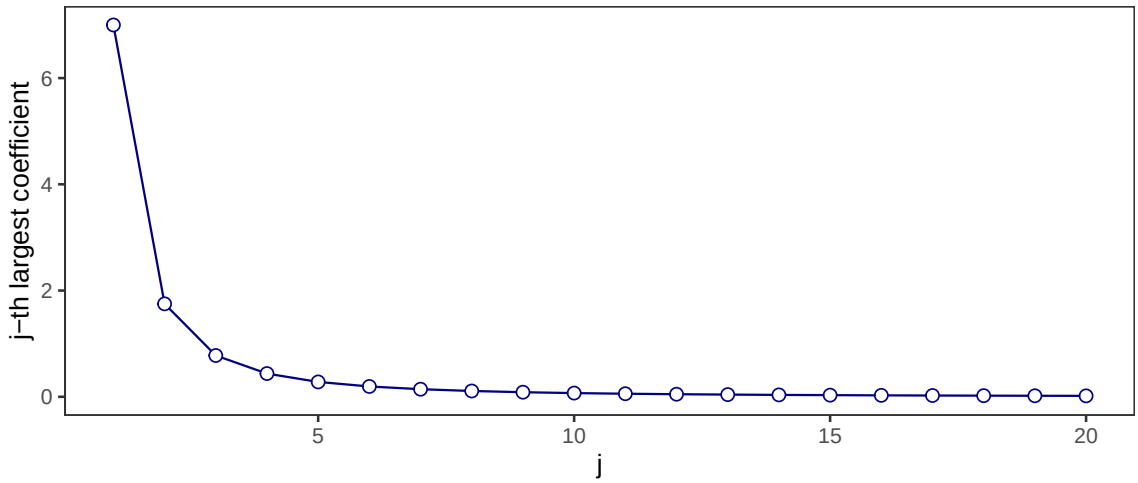


Figure 1: Approximate sparsity: the j-th largest coefficient when $\beta_j = 7/j^2$.

2.3 Lasso and post-lasso

```
## lasso with the penalty chosen by 5-fold cross-validation
fit_lasso_cv <- cv.glmnet(X, y, family = "gaussian", alpha = 1, nfolds = 5)
## lambda.min = the lambda with the smallest cross-validated MSE
lasso_betas <- as.numeric(coef(fit_lasso_cv, s = "lambda.min"))[-1]
sum(lasso_betas != 0)                                  ## how many regressors
kept
#> [1] 7
## post-lasso: OLS on the kept regressors only
kept <- which(lasso_betas != 0)
post_betas <- coef(lm(y ~ X[, kept]))[-1]
## compare the largest few coefficients: truth, lasso, post-lasso
round(rbind(truth = betas[kept], lasso = lasso_betas[kept],
            post_lasso = post_betas), 2)
#>            X[, kept]1 X[, kept]2 X[, kept]3 X[, kept]4 X[, kept]5
#> truth            7.00       1.75       0.00       0.00       0.00
#> lasso            5.90       1.15      -0.02       0.08      -0.35
#> post_lasso       6.32       1.69      -0.43       0.70      -0.75
#>            X[, kept]6 X[, kept]7
#> truth            0.00       0.00
#> lasso           -0.07       0.04
#> post_lasso      -0.59       0.58
```

```
## sort each set of absolute coefficients from largest to smallest and plot
them
sorted <- function(b, lab) data.frame(j = seq_along(b), magnitude =
sort(abs(b), decreasing = TRUE), type = lab)
ggplot(rbind(sorted(betas, "unpenalized (truth)"), sorted(lasso_betas,
"lasso"),
             sorted(post_betas, "post-lasso")),
       aes(j, magnitude, colour = type)) +
  geom_line(linewidth = 0.4) + geom_point(shape = 21, size = 2, fill = "white")
  +
  scale_colour_manual(values = c("unpenalized (truth)" = "navy", "lasso" =
  "firebrick",
                                 "post-lasso" = "darkgreen")) +
  xlim(1, 20) + labs(x = "j", y = "j-th largest coefficient") + theme_lab
```

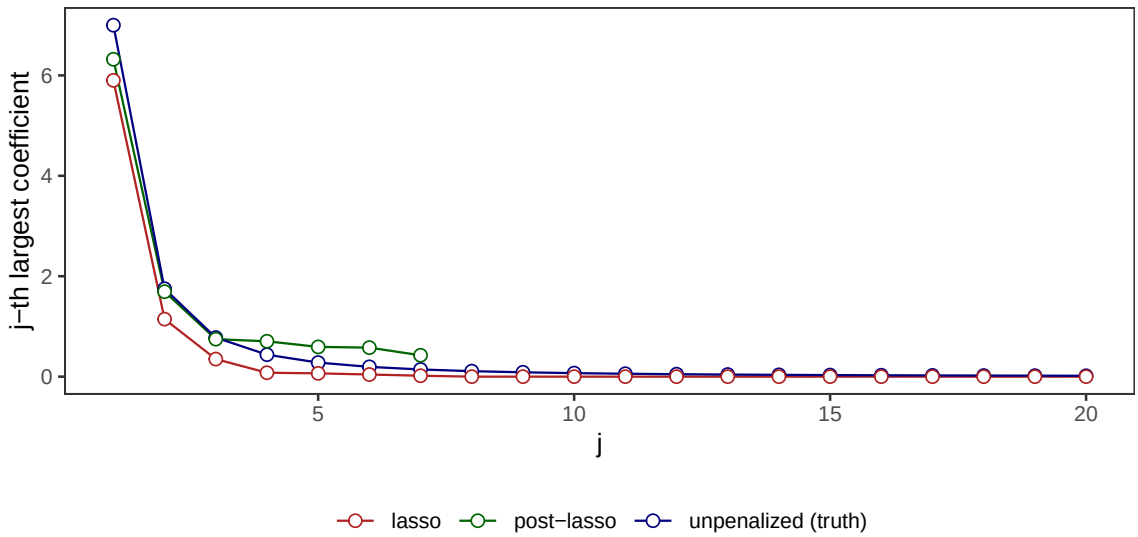


Figure 2: Sorted coefficient sizes: truth, lasso (shrunk toward zero), and post-lasso (shrinkage removed).

The lasso finds the large coefficients but pulls them toward zero (shrinkage bias); post-lasso refits them by OLS and brings them back.

2.4 Ridge, elastic net, and plug-in lasso

```
## out-of-sample R^2, with the training mean as the benchmark
r2_score <- function(pred, actual, ytrain) 1 - sum((actual - pred)^2) /
sum((actual - mean(ytrain))^2)
```

```
## fit every method in one world and return its out-of-sample R^2
race <- function(regime) {
  set.seed(1)
  r <- gen_data(100, 400, regime)
  X <- r$X; y <- r$y; Xp <- r$Xpop; yp <- r$ypop
  lcv <- cv.glmnet(X, y, alpha = 1, nfolds = 5)             ## lasso, CV
  rcv <- cv.glmnet(X, y, alpha = 0, nfolds = 5)             ## ridge, CV
  ecv <- cv.glmnet(X, y, alpha = 0.5, nfolds = 5)           ## elastic net, CV
  rl  <- rlasso(X, y, post = FALSE)                         ## lasso, plug-in
penalty
  rlp <- rlasso(X, y, post = TRUE)                          ## post-lasso,
plug-in
  c(`lasso (CV)`       = r2_score(predict(lcv, Xp, s = "lambda.min"), yp, y),
    `lasso (plug-in)`  = r2_score(predict(rl, newdata = Xp), yp, y),
    `post-lasso (plug-in)` = r2_score(predict(rlp, newdata = Xp), yp, y),
    `ridge (CV)`       = r2_score(predict(rcv, Xp, s = "lambda.min"), yp, y),
    `elastic net (CV)` = r2_score(predict(ecv, Xp, s = "lambda.min"), yp, y))
```

```
}
tab <- sapply(c(sparse = "sparse", dense = "dense", `sparse and dense` =
"sparsedense"), race)
round(tab, 3)
#>                       sparse  dense sparse and dense
#> lasso (CV)             0.779 -0.024            0.335
#> lasso (plug-in)        0.779 -0.065            0.326
#> post-lasso (plug-in)   0.805  0.000            0.283
#> ridge (CV)             0.099  0.077            0.115
#> elastic net (CV)       0.746 -0.056            0.309
```

Read the table by column. In the sparse world the lasso family wins and ridge, spreading weight over 400 terms, fails. In the dense world nothing does well with 100 cases, but ridge is best. When the truth is sparse and dense, the CV lasso, which catches the few big coefficients, does best.

```
## the coefficient shapes of the other two worlds
plot_beta <- function(regime, title, col) {
  set.seed(1); b <- gen_data(100, 400, regime, npop = 10)$beta
  ggplot(data.frame(j = 1:400, magnitude = sort(abs(b), decreasing = TRUE)),
  aes(j, magnitude)) +
    geom_line(colour = col, linewidth = 0.4) +
    geom_point(shape = 21, size = 2, colour = col, fill = "white") +
    xlim(1, 20) + ggtitle(title) + labs(x = "j", y = "j-th largest
    coefficient") + theme_lab
}
ggarrange(plot_beta("dense", "Dense", "navy"), plot_beta("sparsedense", "Sparse
and dense", "firebrick"), nrow = 1)
```

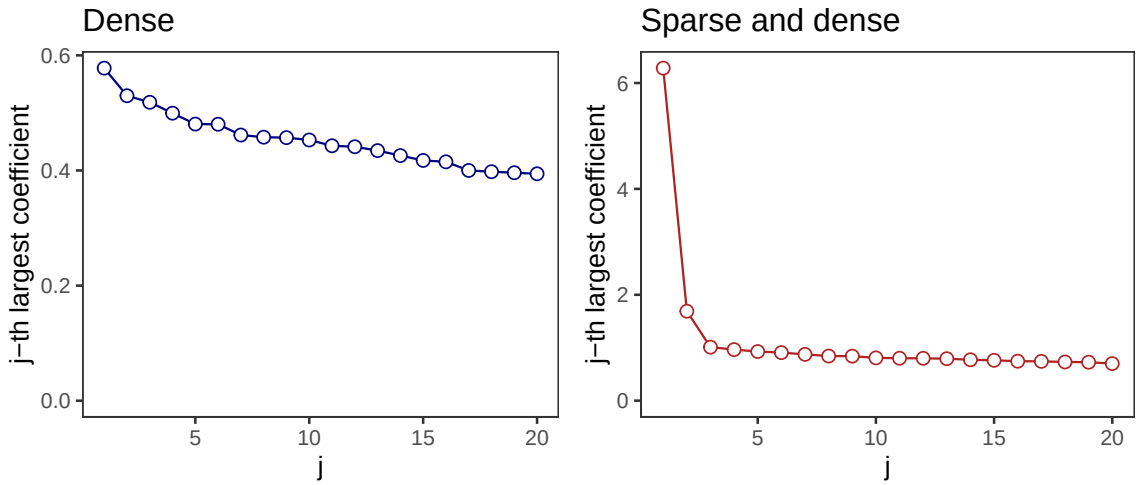


Figure 3: The 20 largest coefficients in the dense and the sparse-and-dense worlds.

3 Lasso on the GSS

3.1 Data, design, and the split

The same steps as `figs/w12_gss.R`, so the numbers match the slides.

```
set.seed(723)
d <- readRDS("../Data/gss_earnings.rds")
## SEI and prestige are left out: both describe the respondent's occupation,
and SEI is
## built partly from occupational earnings, so they would leak the outcome
vars <- c("lnearn", "educ", "age", "female", "black", "otherace", "wordsum",
"pareduc",
          "hours", "childs", "evermar", "region", "year")
d <- d[complete.cases(d[, vars]), ]                    ## complete cases
d$region <- factor(d$region); d$yr <- factor(d$year)
## 23 main-effect columns
f_main <- lnearn ~ educ + age + female + black + otherace + wordsum + pareduc +
  hours + childs + evermar + region + yr
## reason III for large p: all pairwise interactions plus squares
num <- c("educ", "age", "wordsum", "pareduc", "hours", "childs")
f_big <- as.formula(paste("lnearn ~ (educ + age + female + black + otherace +
wordsum + pareduc + hours + childs + evermar + region + yr)^2 +",
                          paste0("I(", num, "^2)", collapse = " + ")))
Xm <- model.matrix(f_main, d)[, -1]
Xb <- model.matrix(f_big, d)[, -1]
Xb <- Xb[, apply(Xb, 2, var) > 0]                      ## drop constant columns
Xb <- Xb[, !duplicated(t(Xb))]                         ## drop duplicated
columns
y <- d$lnearn; n <- nrow(d)
c(n = n, main = ncol(Xm), expanded = ncol(Xb))
#>        n     main expanded
#>     3509       23      243
## 30% test set, never used for fitting or tuning
te <- sample(n, round(0.3 * n)); tr <- setdiff(1:n, te)
r2 <- function(pred) 1 - mean((y[te] - pred)^2) / mean((y[te] - mean(y[tr]))^2)
set.seed(7231)                                         ## same stream as
figs/w12_690s.R
```

3.2 The zero-or-not rule, checked against glmnet

For one standardized regressor with $D = \sum_i X_i^2$, the lasso solution is $(\hat{S} - \lambda)/(2D)$ if $\hat{S} > \lambda$, 0 if $|\hat{S}| \le \lambda$, and $(\hat{S} + \lambda)/(2D)$ if $\hat{S} < -\lambda$, with $\hat{S} = 2\sum_i X_i Y_i$. `glmnet` minimizes $\frac{1}{2n}\sum_i (Y_i - X_i b)^2 + \lambda_g |b|$, so its $\lambda_g = \lambda/(2n)$.

```r
x1 <- as.vector(scale(d$educ[tr])) * sqrt(length(tr) / (length(tr) - 1))  ##
mean 0, sum x^2 = n
y1 <- y[tr] - mean(y[tr])                                  ## centred outcome
S <- 2 * sum(x1 * y1); D <- sum(x1^2)
by_hand <- function(lam) if (S > lam) (S - lam) / (2 * D) else if (S < -lam) (S
+ lam) / (2 * D) else 0
lams <- c(0, 200, 600, 2 * abs(S))                         ## no penalty ... penalty
above |S|
cbind(lambda = lams, OLS = S / (2 * D),
      by_hand = sapply(lams, by_hand),
      glmnet = sapply(lams, function(l) as.numeric(coef(glmnet(cbind(x1, 0),
      y1,
                  lambda = l / (2 * length(tr)), standardize = FALSE, intercept
                  = FALSE))[2])))
#>      lambda    OLS by_hand glmnet
#> [1,]      0 0.3366  0.3366 0.3366
#> [2,]    200 0.3366  0.2959 0.2959
#> [3,]    600 0.3366  0.2145 0.2145
#> [4,]   3307 0.3366  0.0000 0.0000
```

The two columns agree: lasso subtracts $\lambda/(2D)$ from the OLS slope and stops at zero once $\lambda \geq |\hat{S}|$.

3.3 Cross-validation by hand

```r
## 5 folds of the training sample
fold <- sample(rep(1:5, length.out = length(tr)))
path <- glmnet(Xb[tr, ], y[tr], alpha = 1)                 ## one lambda grid for
everything
lams <- path$lambda[c(1, 30, 60)]                          ## a big, a middle, a
small penalty
## for each lambda and each fold: fit without the fold, predict the fold,
record MSE
cvtab <- sapply(lams, function(l) sapply(1:5, function(k) {
  m <- glmnet(Xb[tr[fold != k], ], y[tr[fold != k]], alpha = 1, lambda =
path$lambda)
  mean((y[tr[fold == k]] - predict(m, Xb[tr[fold == k], ], s = l))^2)
}))
nz <- sapply(lams, function(l) sum(coef(path, s = l)[-1] != 0))   ## terms
kept
colnames(cvtab) <- paste0("lambda=", signif(lams, 2), " (", nz, " terms)")
round(rbind(cvtab, `CV-MSE` = colMeans(cvtab)), 3)
```

```
#>        lambda=0.35 (0 terms) lambda=0.024 (42 terms)
#>                        0.853                   0.665
#>                        0.930                   0.708
#>                        0.948                   0.759
#>                        0.784                   0.537
#>                        0.811                   0.591
#> CV-MSE                 0.865                   0.652
#>        lambda=0.0014 (179 terms)
#>                           0.695
#>                           0.748
#>                           0.813
#>                           0.568
#>                           0.670
#> CV-MSE                    0.699
```

The middle penalty wins in every fold. `cv.glmnet` does the same over 100 values:

```
cl <- cv.glmnet(Xb[tr, ], y[tr], alpha = 1, nfolds = 10)
plot(cl)
```

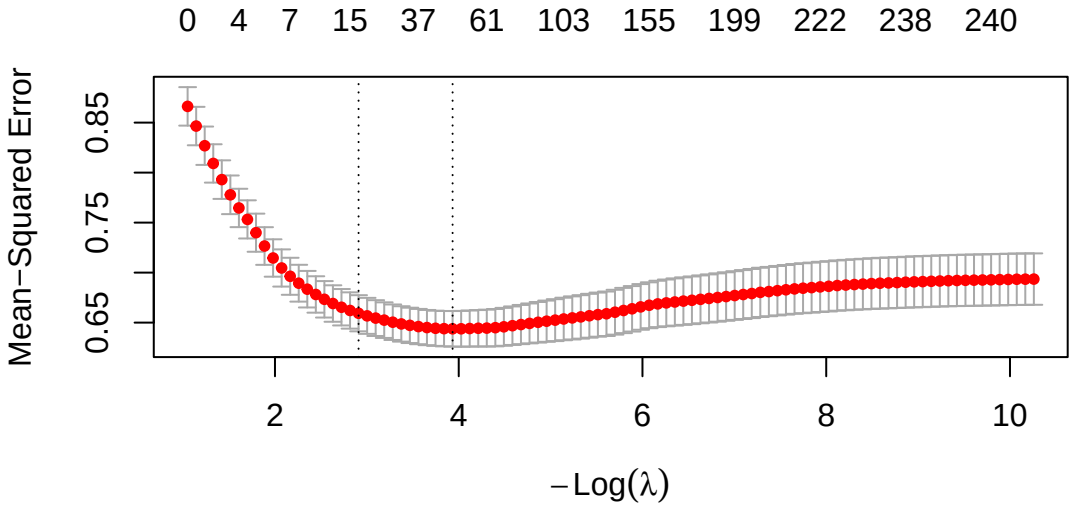


Figure 4: Cross-validated MSE against log penalty. Left dashed line: minimum; right: one-standard-error rule.

```
c(terms_at_min = cl$nzero[cl$lambda == cl$lambda.min],
  terms_at_1se = cl$nzero[cl$lambda == cl$lambda.1se],
  test_R2_min = r2(predict(cl, Xb[te, ], s = "lambda.min")),
  test_R2_1se = r2(predict(cl, Xb[te, ], s = "lambda.1se")))
#> terms_at_min.s31 terms_at_1se.s20      test_R2_min      test_R2_1se
#>          45.0000          17.0000           0.2611           0.2525
```

3.4 The plug-in penalty by hand

$\lambda = 2c\,\hat{\sigma}\sqrt{n}\,z_{1-a/(2p)}$ with $c = 1.1$, $a = 0.1$; start $\hat{\sigma}$ from an intercept-only fit and iterate.

```
ntr <- length(tr); p <- ncol(Xb)
z <- qnorm(1 - 0.1 / (2 * p))                          ## Bonferroni over 2p
tails
Xs <- scale(Xb[tr, ])                                  ## standardized: loadings
= 1
sig <- sd(y[tr])                                       ## step 0: intercept only
for (it in 1:15) {
  lam <- 2 * 1.1 * sig * sqrt(ntr) * z                 ## plug-in lambda
  m <- glmnet(Xs, y[tr], alpha = 1, lambda = lam / (2 * ntr), standardize =
FALSE)
  sig_new <- sqrt(mean((y[tr] - predict(m, Xs))^2))    ## update sigma from the
fit
  cat(sprintf("iter %d: sigma %.3f -> lambda %.1f, terms kept %d\n",
              it, sig, lam, sum(coef(m)[-1] != 0)))
  if (abs(sig_new - sig) < 1e-4) break
  sig <- sig_new
}
#> iter 1: sigma 0.931 -> lambda 358.7, terms kept 14
#> iter 2: sigma 0.814 -> lambda 313.4, terms kept 14
#> iter 3: sigma 0.810 -> lambda 311.9, terms kept 14
#> iter 4: sigma 0.810 -> lambda 311.9, terms kept 14
## test R^2 of the plug-in lasso
Xte_s <- scale(Xb[te, ], attr(Xs, "scaled:center"), attr(Xs, "scaled:scale"))
r2(as.vector(predict(m, Xte_s)))
#> [1] 0.2477
```

`hdm::rlasso` runs the same idea with refinements (data-driven loadings), so its selection need not match ours exactly:

```
rl <- rlasso(Xb[tr, ], y[tr], post = FALSE)
c(terms_kept = sum(coef(rl)[-1] != 0), test_R2 = r2(predict(rl, newdata =
Xb[te, ])))
#> terms_kept    test_R2
#>    27.0000     0.2592
```

3.5 Post-lasso

```
## 23 main effects, standardized, lasso at the 1-se lambda, then OLS on the
kept ones
Zm <- scale(Xm[tr, ])
```

```r
clm <- cv.glmnet(Zm, y[tr], alpha = 1, nfolds = 10)
bl <- coef(clm, s = "lambda.1se")[-1]; names(bl) <- colnames(Xm)
km <- which(bl != 0)
bp <- coef(lm(y[tr] ~ Zm[, km]))[-1]
round(rbind(lasso = bl[km], post_lasso = bp), 3)
#>             educ   age female  black wordsum pareduc hours evermar yr2022
#> lasso      0.251 0.103 -0.121 -0.017   0.043   0.024 0.050   0.058  0.028
#> post_lasso 0.286 0.155 -0.174 -0.058   0.061   0.070 0.101   0.098  0.087
## on the 243-term design: does removing the shrinkage help prediction?
keep_plugin <- which(coef(m)[-1] != 0)
post_plugin <- lm.fit(cbind(1, Xb[tr, keep_plugin]), y[tr])
c(plugin_lasso = r2(as.vector(predict(m, Xte_s))),
  plugin_post_lasso = r2(as.vector(cbind(1, Xb[te, keep_plugin]) %*%
  coef(post_plugin))))
#>      plugin_lasso plugin_post_lasso
#>            0.2477            0.2590
```

3.6 Lasso selection is unstable

```r
## refit the lasso on 100 bootstrap samples; how often is each variable kept?
sel <- replicate(100, {
  b <- sample(tr, replace = TRUE)
  cvb <- cv.glmnet(Xm[b, ], y[b], alpha = 1, nfolds = 5)
  as.vector(coef(cvb, s = "lambda.1se"))[-1] != 0
})
freq <- rowMeans(sel); names(freq) <- colnames(Xm)
round(sort(freq, decreasing = TRUE)[1:12], 2)
#>           educ            age         female       wordsum        evermar
#>           1.00           1.00           1.00           1.00           1.00
#>          hours         yr2022        pareduc          black  regionPacific
#>           0.97           0.88           0.87           0.77           0.39
#> regionMountain         yr2014
#>           0.30           0.13
## children as a main effect, and the female x children interaction in the
plug-in lasso
c(children_main_effect = freq[["childs"]],
  plugin_keeps_female_x_children = "female:childs" %in%
  colnames(Xb)[keep_plugin])
#>           children_main_effect plugin_keeps_female_x_children
#>                           0.02                           1.00
```

> **Important**
>
> The lasso rarely keeps the number of children as a main effect, yet the plug-in lasso on the expanded design keeps female × children: children matter through sex. Weaker predictors flip in and out across samples. **The selected set is not a ranking of importance**, and p-values from a naive refit on the selected set are not valid.

3.7 How to do cross-validation wrong

```
## WRONG: screen the 40 columns most correlated with y on ALL the data, then
CV
cors <- apply(Xb, 2, function(z) abs(cor(z, y)))
top <- order(cors, decreasing = TRUE)[1:40]
cv_leak <- cv.glmnet(Xb[, top], y, alpha = 1, nfolds = 10)
## RIGHT: screen inside each fold, using only that fold's training cases
folds <- sample(rep(1:10, length.out = n))
honest <- sapply(1:10, function(k) {
  a <- which(folds != k); b <- which(folds == k)
  cc <- apply(Xb[a, ], 2, function(z) abs(cor(z, y[a])))
  tp <- order(cc, decreasing = TRUE)[1:40]
  fit <- cv.glmnet(Xb[a, tp], y[a], alpha = 1, nfolds = 5)
  mean((y[b] - predict(fit, Xb[b, tp], s = "lambda.min"))^2)
})
c(leaky_cv_mse = min(cv_leak$cvm), honest_cv_mse = mean(honest))
#>  leaky_cv_mse honest_cv_mse
#>        0.6430        0.6497
```

> **Warning**
>
> **Every data-dependent choice must happen inside the fold**: screening, standardization, imputation, and tuning alike. Here the leak is small because 2,456 cases dilute it; with $p \gg n$ it can be enormous.

4 Trees and ensembles

4.1 A noiseless curve (690S)

```
## Y = exp(4X), no noise: the only error is approximation error
set.seed(1)
x_train <- matrix(runif(1000), 1000, 1)
y_train <- as.vector(exp(4 * x_train))
fitml <- data.frame(x = x_train[, 1], y = y_train)
```

```
## plot the truth (blue) and a method's predictions (red)
plot_fit <- function(pred, title) {
  ggplot(data.frame(x = fitml$x, y = fitml$y, pred = pred), aes(x)) +
    geom_point(aes(y = y), size = 0.5, colour = "navy") +
    geom_point(aes(y = pred), size = 0.5, colour = "firebrick", alpha = 0.6) +
    ggtitle(title) + labs(x = "X", y = "g(X)") + theme_lab
}
```

```
shallow <- rpart(y ~ x, data = fitml, cp = 0.01)     ## cp = complexity
penalty
deep    <- rpart(y ~ x, data = fitml, cp = 0.0005)
ggarrange(plot_fit(predict(shallow), "shallow tree"), plot_fit(predict(deep),
"deep tree"), nrow = 1)
c(leaves_shallow = sum(shallow$frame$var == "<leaf>"), leaves_deep =
sum(deep$frame$var == "<leaf>"))
#> leaves_shallow    leaves_deep
#>              6             16
```

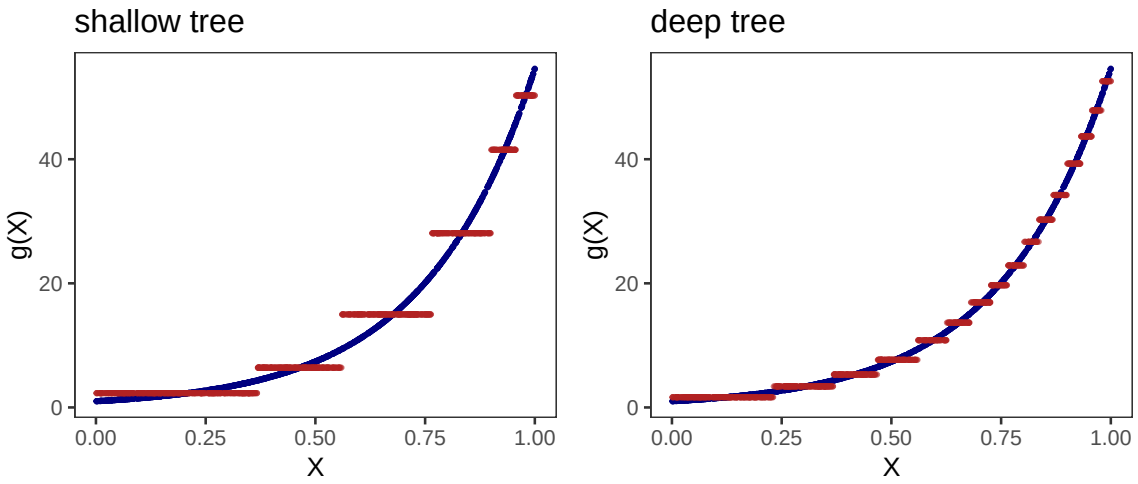


Figure 5: A shallow tree (complexity penalty cp = .01) and a deep tree (cp = .0005).

```
rpart.plot(shallow, type = 2, extra = 1, under = TRUE, fallen.leaves = TRUE)
```

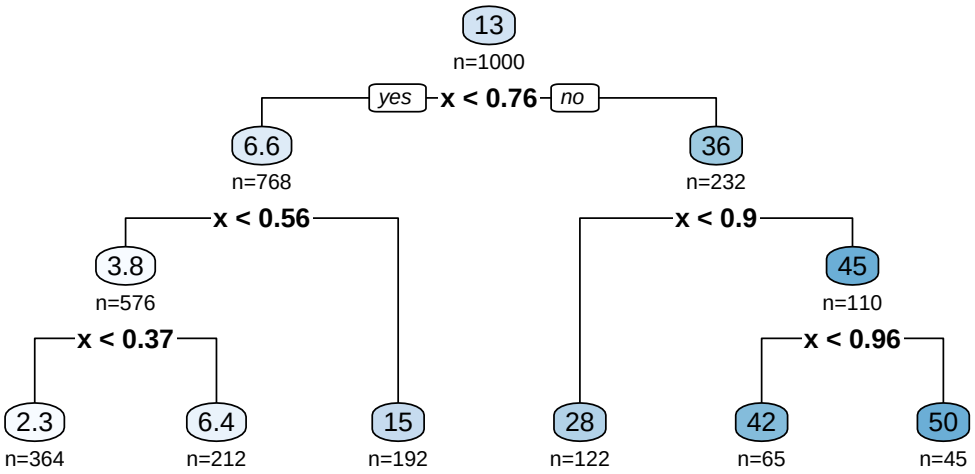


Figure 6: The shallow tree drawn as a tree: each leaf predicts the mean of its cases.

4.2 One split by hand

```
toy <- data.frame(educ = c(10, 12, 12, 14, 16, 18),
                  y    = c(9.6, 9.8, 10.0, 10.4, 10.8, 11.0))
sse <- function(v) sum((v - mean(v))^2)              ## within-node sum of
squares
## try every cut: SSE left + SSE right
t(sapply(c(11, 13, 15, 17), function(s) {
  L <- toy$y[toy$educ <= s]; R <- toy$y[toy$educ > s]
  c(cut = s, n_left = length(L), mean_left = mean(L), mean_right = mean(R),
    SSE = sse(L) + sse(R))
}))
#>      cut n_left mean_left mean_right    SSE
#> [1,]  11      1      9.60      10.40 1.0400
#> [2,]  13      3      9.80      10.73 0.2667
#> [3,]  15      4      9.95      10.90 0.3700
#> [4,]  17      5     10.12      11.00 0.9280
## rpart finds the same cut (it reports the midpoint between 12 and 14)
rpart(y ~ educ, toy, control = rpart.control(minsplit = 2, minbucket = 1,
maxdepth = 1, cp = 0))
#> n= 6
#>
#> node), split, n, deviance, yval
#>       * denotes terminal node
#>
#> 1) root 6 1.5730 10.27
#>   2) educ< 13 3 0.0800  9.80 *
#>   3) educ>=13 3 0.1867 10.73 *
```

4.3 Trees on the GSS

```
dtr <- data.frame(y = y[tr], Xm[tr, ]); dte <- data.frame(Xm[te, ])
## the first split rpart chooses, and a depth-3 tree
t3 <- rpart(y ~ ., dtr, control = rpart.control(maxdepth = 3, cp = 0))
t3
#> n= 2456
#>
#> node), split, n, deviance, yval
#>       * denotes terminal node
#>
#>  1) root 2456 2130.00  9.937
#>    2) educ< 15.5 1407 1138.00  9.657
#>      4) female>=0.5 651  489.90  9.428
#>        8) age< 39.5 277  229.50  9.183 *
#>        9) age>=39.5 374  231.50  9.609 *
#>      5) female< 0.5 756  583.90  9.855
#>       10) evermar< 0.5 211  160.90  9.503 *
#>       11) evermar>=0.5 545  386.70  9.991 *
#>    3) educ>=15.5 1049  733.00 10.310
#>      6) age< 32.5 200  100.20  9.943
#>       12) age< 25.5 9   12.20  9.036 *
#>       13) age>=25.5 191   80.27  9.986 *
#>      7) age>=32.5 849  598.90 10.400
#>       14) female>=0.5 445  278.60 10.230 *
#>       15) female< 0.5 404  291.50 10.590 *
## depth as the bias-variance dial: training vs test R^2
t(sapply(c(1, 2, 3, 5, 8, 12, 20), function(dep) {
  tt <- rpart(y ~ ., dtr, control = rpart.control(maxdepth = dep, cp = 0,
minsplit = 2, minbucket = 1))
  c(depth = dep, leaves = sum(tt$frame$var == "<leaf>"),
    train_R2 = 1 - mean((y[tr] - predict(tt, dtr))^2) / mean((y[tr] -
    mean(y[tr]))^2),
    test_R2 = r2(predict(tt, dte)))
})) |> round(3)
#>      depth leaves train_R2 test_R2
#> [1,]     1      2    0.122   0.132
#> [2,]     2      4    0.168   0.179
#> [3,]     3      8    0.215   0.231
#> [4,]     5     31    0.313   0.218
#> [5,]     8    180    0.540   0.074
#> [6,]    12    708    0.769  -0.101
#> [7,]    20   1938    0.975  -0.368
```

```
rpart.plot(t3, type = 2, extra = 1, under = TRUE, fallen.leaves = TRUE, cex =
0.6)
```

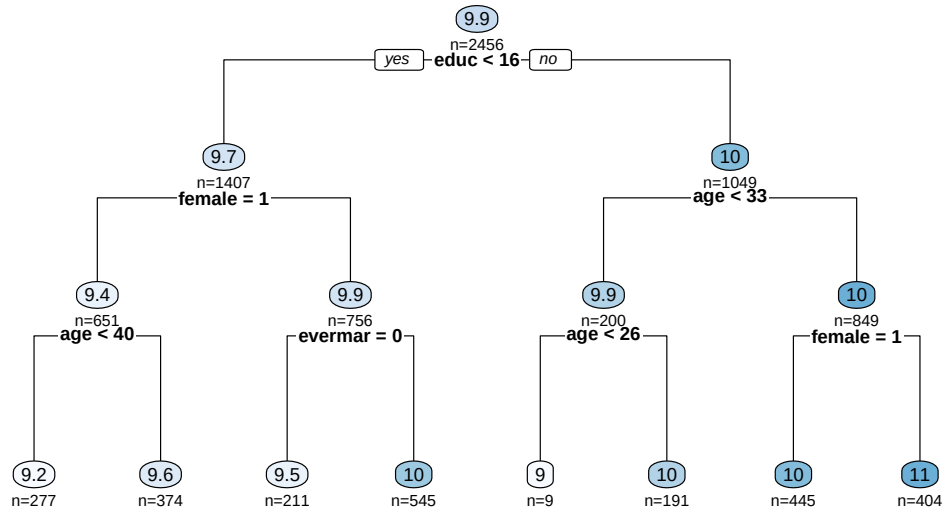


Figure 7: The depth-3 tree on the GSS. Below 16 years of schooling the tree splits next on sex; above, on age: an interaction.

4.4 Bagging and random forests

```
## 690S code: randomForest grows 500 trees; leaves of at most 5 cases
RFmodel <- randomForest(y ~ x, data = fitml, ntree = 500, nodesize = 5)
## boosted trees: n.trees = number of steps, shrinkage = learning rate, depth
3
BoostTreemodel <- gbm(y ~ x, data = fitml, distribution = "gaussian",
                      n.trees = 100, shrinkage = 0.05, interaction.depth = 3)
ggarrange(plot_fit(predict(RFmodel, fitml), "random forest"),
          plot_fit(predict(BoostTreemodel, fitml, n.trees = 100), "boosted
          trees"), nrow = 1)
```

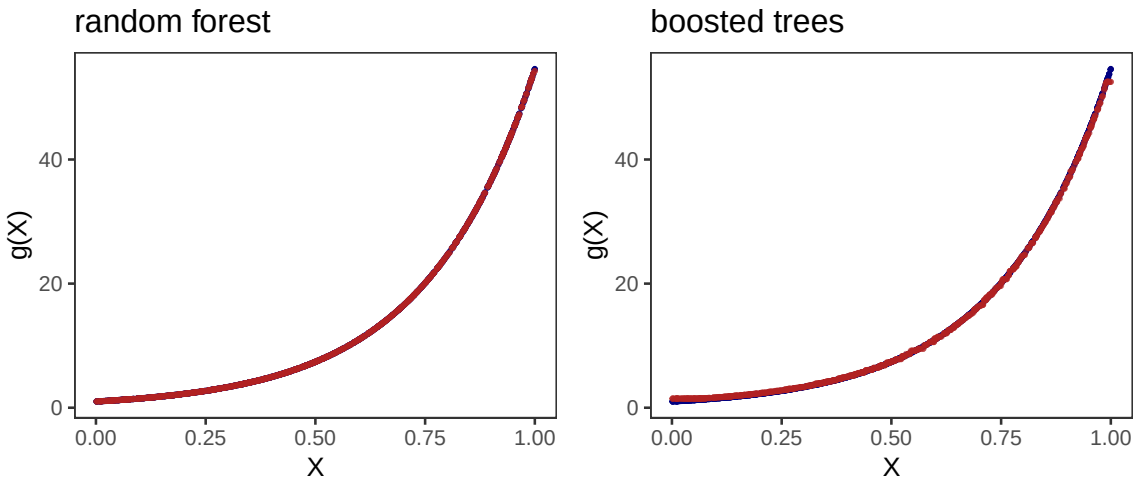


Figure 8: Random forest (500 trees) and boosted trees (100 steps, learning rate .05) on the noiseless curve.

On the GSS: one deep tree, 500 bagged deep trees (every split may use all 23 predictors), and a random forest (each split sees 5 random predictors).

```
deep1 <- ranger(y ~ ., dtr, num.trees = 1, mtry = ncol(Xm), replace = FALSE,
                sample.fraction = 1, min.node.size = 5, seed = 1)
bag <- ranger(y ~ ., dtr, num.trees = 500, mtry = ncol(Xm), min.node.size = 5,
seed = 1)
rf5 <- ranger(y ~ ., dtr, num.trees = 500, mtry = 5, min.node.size = 5, seed =
1)
c(one_deep_tree = r2(predict(deep1, dte)$predictions),
  bagging = r2(predict(bag, dte)$predictions),
  random_forest = r2(predict(rf5, dte)$predictions))
#> one_deep_tree       bagging random_forest
#>       -0.2643        0.2679        0.2737
## how correlated are two trees? (average over pairs among the first 50 trees)
avg_cor <- function(fit) { cm <- cor(predict(fit, dte, predict.all =
TRUE)$predictions[, 1:50]); mean(cm[upper.tri(cm)]) }
c(bagging = avg_cor(bag), random_forest = avg_cor(rf5))
#>       bagging random_forest
#>        0.3418        0.2904
## out-of-bag MSE: a free estimate of test MSE
c(oob_mse = rf5$prediction.error, test_mse = mean((y[te] - predict(rf5,
dte)$predictions)^2))
#>  oob_mse test_mse
#>   0.6262   0.6438
```

4.5 Boosting by hand

```
## best one-split tree (a stump) for residuals r on x
stump <- function(x, r) {
  cuts <- sort(unique(x)); cuts <- (head(cuts, -1) + tail(cuts, -1)) / 2
  s <- cuts[which.min(sapply(cuts, function(c) sse(r[x <= c]) + sse(r[x >
c])))]
  list(cut = s, fit = ifelse(x <= s, mean(r[x <= s]), mean(r[x > s])))
}
R <- toy$y - mean(toy$y)                              ## start from the mean
lambda <- 0.5                                         ## learning rate
for (j in 1:4) {
  st <- stump(toy$educ, R)                            ## fit a stump to the
residuals
  R <- R - lambda * st$fit                            ## subtract a fraction of
it
  cat(sprintf("step %d: cut %.0f, residuals %s, SSR %.3f\n", j, st$cut,
              paste(round(R, 2), collapse = " "), sum(R^2)))
}
#> step 1: cut 13, residuals -0.43 -0.23 -0.03 -0.1 0.3 0.5, SSR 0.593
#> step 2: cut 15, residuals -0.33 -0.13 0.07 0 0.1 0.3, SSR 0.233
#> step 3: cut 11, residuals -0.17 -0.17 0.03 -0.03 0.07 0.27, SSR 0.133
#> step 4: cut 17, residuals -0.14 -0.14 0.06 -0.01 0.09 0.13, SSR 0.069
```

The same loop on the GSS, with depth-3 trees and a learning rate of 0.05:

```
boost <- function(M = 400, eta = 0.05, depth = 3) {
  f_tr <- rep(mean(y[tr]), length(tr)); f_te <- rep(mean(y[tr]), length(te))
  dd <- dtr
  for (m in seq_len(M)) {
    dd$y <- y[tr] - f_tr                              ## current residuals
    t_m <- rpart(y ~ ., dd, control = rpart.control(maxdepth = depth, cp = 0,
xval = 0))
    f_tr <- f_tr + eta * predict(t_m, dd)             ## add a shrunken tree
    f_te <- f_te + eta * predict(t_m, dte)
  }
  f_te
}
c(steps_100 = r2(boost(100)), steps_400 = r2(boost(400)))
#> steps_100 steps_400
#>    0.2819    0.2654
```

5 Neural networks

5.1 Gradient descent and backpropagation by hand

```
## minimize R(theta) = 1/2 sum (Y - theta)^2 by gradient descent, rho = 0.1
Y <- c(1, 2, 6); theta <- 0
for (t in 0:5) {
  grad <- -sum(Y - theta)                              ## dR/dtheta
  cat(sprintf("t %d: theta %.3f, R %.3f, gradient %.3f\n", t, theta, 0.5 *
  sum((Y - theta)^2), grad))
  theta <- theta - 0.1 * grad                          ## step downhill
}
#> t 0: theta 0.000, R 20.500, gradient -9.000
#> t 1: theta 0.900, R 13.615, gradient -6.300
#> t 2: theta 1.530, R 10.241, gradient -4.410
#> t 3: theta 1.971, R 8.588, gradient -3.087
#> t 4: theta 2.280, R 7.778, gradient -2.161
#> t 5: theta 2.496, R 7.381, gradient -1.513
```

```
## one case, one ReLU hidden unit: forward, backward, update
X <- 1; Yc <- 4; w0 <- 0; w1 <- 2; b0 <- 0; b1 <- 1.5; rho <- 0.1
z <- w0 + w1 * X; A <- max(0, z); f <- b0 + b1 * A; r <- Yc - f   ## forward
pass
g_b1 <- -r * A                                         ## dR/db1
g_w1 <- -r * b1 * (z > 0) * X                          ## dR/dw1, ReLU slope = 1
here
b1 <- b1 - rho * g_b1; w1 <- w1 - rho * g_w1           ## update
f_new <- b0 + b1 * max(0, w0 + w1 * X)
c(residual = r, dR_db1 = g_b1, dR_dw1 = g_w1, b1 = b1, w1 = w1,
  f_new = f_new, loss_before = 0.5 * r^2, loss_after = 0.5 * (Yc - f_new)^2)
#>    residual      dR_db1      dR_dw1          b1          w1       f_new
#>     1.00000    -2.00000    -1.50000     1.70000     2.15000     3.65500
#> loss_before  loss_after
#>     0.50000     0.05951
```

5.2 A network on the noiseless curve

```
ys <- (y_train - mean(y_train)) / sd(y_train)          ## networks like
standardized targets
n5   <- nnet(x_train, ys, size = 10, linout = TRUE, maxit = 5,   trace = FALSE)
n500 <- nnet(x_train, ys, size = 10, linout = TRUE, maxit = 500, trace = FALSE)
back <- function(m) mean(y_train) + sd(y_train) * as.vector(predict(m,
x_train))
```

```
ggarrange(plot_fit(back(n5), "5 iterations"), plot_fit(back(n500), "500
iterations"), nrow = 1)
```

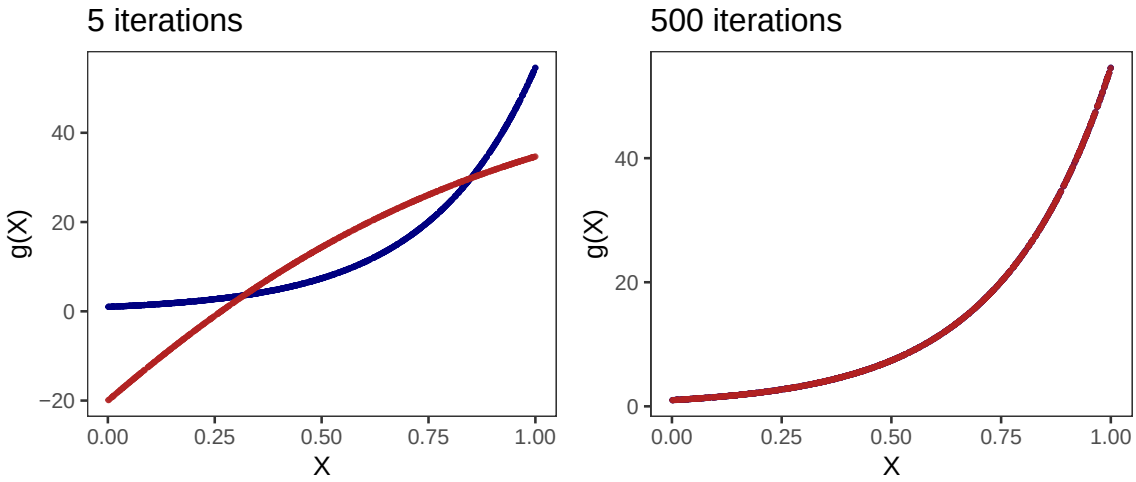


Figure 9: One hidden layer of 10 units, stopped after 5 or 500 iterations.

The 690S code fits the same curve with `keras3` (two hidden layers of 200 and 20 ReLU units, Adam, mini-batches of 10, early stopping on a 20% validation set). It needs a Python installation of TensorFlow, so it is shown but not run here:

```
library(keras3)   ## install_keras(tensorflow = "default") once
build_model <- function() {
  keras_model_sequential() |>
    layer_dense(units = 200, activation = "relu", input_shape = c(1)) |>
    layer_dense(units = 20, activation = "relu") |>
    layer_dense(units = 1) |>
    compile(optimizer = optimizer_adam(learning_rate = 0.01), loss = "mse")
}
## one epoch vs 100 epochs, mini-batches of 10
model1 <- build_model();   model1   |> fit(x_train, y_train, epochs = 1,
batch_size = 10, verbose = 0)
model100 <- build_model(); model100 |> fit(x_train, y_train, epochs = 100,
batch_size = 10, verbose = 0)
## early stopping: stop when the validation loss has not improved for 5 epochs
early <- callback_early_stopping(monitor = "val_loss", patience = 5)
model <- build_model()
model |> fit(x_train, y_train, epochs = 100, batch_size = 10,
             validation_split = 0.2, callbacks = list(early), verbose = 0)
```

5.3 A network on the GSS

```
## standardize inputs with the training means and sds
mu <- colMeans(Xm[tr, ]); sdv <- apply(Xm[tr, ], 2, sd)
Ztr <- scale(Xm[tr, ], mu, sdv); Zte <- scale(Xm[te, ], mu, sdv)
ym <- mean(y[tr]); ysd <- sd(y[tr])
## the size and weight decay chosen by CV in figs/w12_gss.R: 2 units, decay
0.1
## average 5 random starts (the loss is not convex)
pred <- rowMeans(replicate(5, {
  m <- nnet(Ztr, (y[tr] - ym) / ysd, size = 2, decay = 0.1, linout = TRUE,
maxit = 500, trace = FALSE)
  ym + ysd * as.vector(predict(m, Zte))
}))
c(parameters = (ncol(Xm) + 1) * 2 + 2 + 1, test_R2 = r2(pred))
#> parameters    test_R2
#>    51.0000     0.2543
```

6 Exercises

1. **Which penalty wins.** In the simulation, change n from 100 to 400 in all three worlds. Which methods improve most, and why does the dense world gain the most?
2. **The zero-or-not rule.** Using the `soft-threshold` chunk, find the smallest λ at which education drops out. Show it equals $|\hat{S}|$.
3. **Post-lasso.** Why do all nine post-lasso coefficients exceed the lasso ones in size? When would post-lasso predict worse than lasso?
4. **Depth.** Grow trees of depth 1 to 20 on the GSS with `minbucket = 20` instead of 1. How do the training and test R^2 change? Explain with bias and variance.
5. **Forests.** Vary `mtry` over 2, 5, 10, 23 for `ranger`. Report the test R^2 and the average correlation between trees. Is lower correlation always better?
6. **Boosting's dials.** In `boost()`, try $\eta \in \{0.01, 0.1, 0.5\}$ with $M \in \{100, 400\}$. Show that small η with many steps beats large η with few, in terms of bias.
7. **Prediction is not inference.** Take the post-lasso coefficient on `female`. Explain in two sentences why you would not report it with the usual standard error.

7 Session information

```
## R version and the versions of the packages loaded above
cat(R.version.string, "\n")
#> R version 4.5.2 (2025-10-31)
sapply(sessionInfo()$otherPkgs, `[[`, "Version")
```

```
#>         nnet          gbm randomForest       ranger   rpart.plot
#>     "7.3-20"      "2.3.1"    "4.7-1.2"     "0.18.0"      "3.1.5"
#>        rpart       ggpubr      ggplot2          hdm       glmnet
#>     "4.1.24"      "0.6.3"      "4.0.2"      "0.3.2"        "5.1"
#>       Matrix
#>      "1.7-4"
```